Java Backend Architecture

Interview Series

Chapter 7.6
Gradle Dependency Management

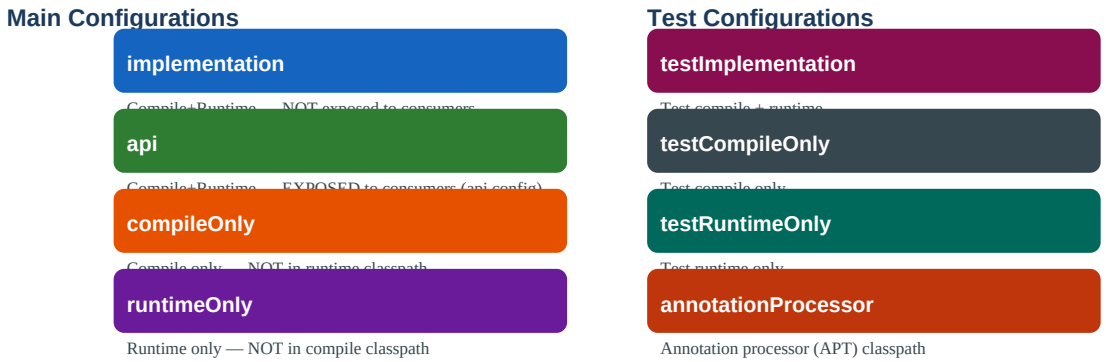
50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 7.6 — Gradle Dependency Management

Introduction

Gradle's dependency management system is more expressive than Maven's. It offers fine-grained configurations (implementation, api, compileOnly, runtimeOnly), version catalogs for centralised version management, platform/BOM support, dependency locking for reproducible builds, rich version constraints, and powerful conflict resolution strategies.

Dependency Configurations



implementation vs api: api leaks the dependency to consumers of this library — use only for public API types

```
// build.gradle.kts dependencies { // implementation: on compile+runtime CP, NOT exposed to
consumers implementation("org.springframework.boot:spring-boot-starter-web:3.2.0") // api:
compile+runtime CP, EXPOSED to consumers (requires java-library plugin)
api("com.fasterxml.jackson.core:jackson-databind:2.16.1") // compileOnly: only compile CP
(like Maven provided) compileOnly("org.projectlombok:lombok:1.18.30")
annotationProcessor("org.projectlombok:lombok:1.18.30") // runtimeOnly: only runtime CP (like
Maven runtime) runtimeOnly("org.postgresql:postgresql:42.7.1") // test scopes
testImplementation("org.junit.jupiter:junit-jupiter:5.10.1")
testRuntimeOnly("org.junit.platform:junit-platform-launcher")
testCompileOnly("org.projectlombok:lombok:1.18.30")
testAnnotationProcessor("org.projectlombok:lombok:1.18.30") }
```

implementation vs api — Critical Difference

This is one of the most important Gradle concepts for library authors. **implementation** hides the dependency from consumers — they cannot compile against it transitively. **api** exposes the dependency — consumers get it on their compile classpath. Use api only when consumers MUST use types from the dependency in your public API. Requires the **java-library** plugin.

```
plugins { `java-library` // enables api configuration } dependencies { // Use api only when the
type appears in your public method signatures api("com.google.guava:guava:32.1.3-jre") // e.g.,
ImmutableList in return type // Use implementation for internal-only dependencies
implementation("org.apache.commons:commons-lang3:3.14.0") // internal only }
```

Rich Version Constraints

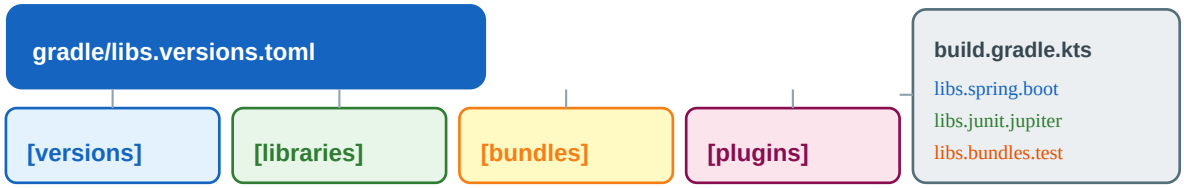
```
dependencies { implementation("org.springframework.boot:spring-boot-starter-web") { version {
// require: minimum acceptable version require("3.1.0") // prefer: hint when multiple options
```

```
satisfy constraints prefer("3.2.0") // strictly: hard ceiling — fail if anything tries to use higher // strictly("[3.0,3.3)") // reject: blacklist specific broken versions reject("3.1.1") // known buggy version } } }
```

Excluding Transitive Dependencies

```
dependencies { implementation("org.springframework.boot:spring-boot-starter-logging") { // Exclude logback, using log4j2 instead exclude(group = "ch.qos.logback", module = "logback-classic") } // Global exclusion for ALL configurations: configurations.all { exclude(group = "commons-logging", module = "commons-logging") } }
```

Version Catalogs (libs.versions.toml)



Version catalog provides type-safe accessors — libs.spring.boot instead of "org.springframework.boot:spring-boot-starter:3.2.0"
Central version management — update spring.version once, affects all libraries using it

```
# gradle/libs.versions.toml [versions] spring-boot = "3.2.0" jackson = "2.16.1" junit = "5.10.1" testcontainers = "1.19.3" [libraries] spring-boot-web = { module = "org.springframework.boot:spring-boot-starter-web", version.ref = "spring-boot" } spring-boot-test = { module = "org.springframework.boot:spring-boot-starter-test", version.ref = "spring-boot" } jackson-databind = { module = "com.fasterxml.jackson.core:jackson-databind", version.ref = "jackson" } junit-jupiter = { module = "org.junit.jupiter:junit-jupiter", version.ref = "junit" } testcontainers-postgresql = { module = "org.testcontainers:postgresql", version.ref = "testcontainers" } [bundles] testing = ["junit-jupiter", "testcontainers-postgresql", "spring-boot-test"] [plugins] spring-boot = { id = "org.springframework.boot", version.ref = "spring-boot" }
```

```
// build.gradle.kts — using type-safe accessors plugins { alias(libs.plugins.spring.boot) } dependencies { implementation(libs.spring.boot.web) implementation(libs.jackson.databind) // bundle: adds all listed dependencies at once testImplementation(libs.bundles.testing) }
```

Platform / BOM Support

```
// Import a BOM (Maven-style) as a platform dependencies { // enforcedPlatform: versions are strictly enforced (cannot be overridden) implementation(enforcedPlatform("org.springframework.boot:spring-boot-dependencies:3.2.0")) // platform: soft constraint (can be overridden by explicit version) implementation(platform("io.micrometer:micrometer-bom:1.12.0")) // After importing platform, declare without version: implementation("org.springframework.boot:spring-boot-starter-web") implementation("io.micrometer:micrometer-core") }
```

Dependency Locking

```
// Enable dependency locking for reproducible builds dependencyLocking { lockAllConfigurations() lockMode = LockMode.STRICT // fail if deps not in lock file } # Generate/update the lock file: ./gradlew dependencies --write-locks # Lock file: gradle/dependency-locks/compileClasspath.lockfile # Contents: # org.springframework.boot:spring-boot-starter-web:3.2.0=compileClasspath,runtimeClasspath # ch.qos.logback:logback-classic:1.4.14=runtimeClasspath # Update specific dependency: ./gradlew
```

```
dependencies --update-locks org.springframework.boot:spring-boot-starter-web
```

Conflict Resolution

```
// Default: highest version wins (unlike Maven's nearest-wins) configurations.all {
resolutionStrategy { // Force a specific version regardless of what deps request
force("org.slf4j:slf4j-api:2.0.9") // Fail build on version conflict (for strict control)
failOnVersionConflict() // Substitute one module with another dependencySubstitution {
substitute(module("commons-logging:commons-logging"))
.using(module("org.slf4j:jcl-over-slf4j:2.0.9")) } // Cache dynamic versions for 10 minutes
cacheDynamicVersionsFor(10, "minutes") // Cache changing (SNAPSHOT) modules for 0 seconds
(always check) cacheChangingModulesFor(0, "seconds") } }
```

50+ Interview Questions & Answers

Q1. What is the difference between implementation and api in Gradle?

implementation hides the dependency from consumers — not on their compile classpath. api exposes it to consumers. Use api only when consumers need to compile against types from your dependency. Requires java-library plugin.

Q2. Why should you prefer implementation over api?

implementation gives better build performance — changes to the dependency don't require recompiling consumers. api leaks the dependency, causing unnecessary recompilation cascades in multi-project builds.

Q3. What is the java-library plugin?

Extends the java plugin with the api configuration. Required when you're building a library (not an application) that other modules depend on. Applications use the java plugin which only provides implementation.

Q4. What is compileOnly scope in Gradle?

The dependency is on the compile classpath only — not in the runtime JAR. Used for Lombok (annotation processing), provided APIs like servlet-api, or compile-time-only annotations.

Q5. What is runtimeOnly scope in Gradle?

Available at runtime but not on the compile classpath. Used for JDBC drivers (loaded reflectively), logging implementations (log4j2 when compiling against SLF4J API only).

Q6. What is annotationProcessor configuration?

Puts the JAR on the annotation processor path (javac -processorpath), not the compile classpath. Required for Lombok, MapStruct, Dagger, and other APT-based code generators.

Q7. What is a version catalog?

A libs.versions.toml file in the gradle/ directory providing a centralized, type-safe way to declare dependency versions. Accessed as libs.spring.boot.web instead of string coordinates.

Q8. What are the four sections of libs.versions.toml?

[versions] — version strings. [libraries] — dependency coordinates referencing versions. [bundles] — groups of libraries. [plugins] — plugin IDs with versions.

Q9. What is a bundle in version catalogs?

A named group of library aliases that can be added as a single dependency: testImplementation(libs.bundles.testing) adds all libraries listed in the bundle.

Q10. How do you reference a version catalog library in build.gradle.kts?

Use the generated type-safe accessor: implementation(libs.spring.boot.web). The accessor is generated from the TOML key with dashes converted to dots.

Q11. What is the difference between platform() and enforcedPlatform()?

platform() imports BOM versions as recommendations — they can be overridden by explicit version declarations. enforcedPlatform() strictly enforces versions — explicit overrides are rejected.

Q12. What does Gradle's conflict resolution strategy default to?

Highest version wins — unlike Maven's nearest-wins. If multiple versions of the same dependency are requested, Gradle selects the highest version by default.

Q13. How do you force a specific version in Gradle?

configurations.all { resolutionStrategy { force('org.slf4j:slf4j-api:2.0.9') } }. Or use a rich version strictly constraint in the dependency declaration.

Q14. What is dependency substitution?

Replacing one module with another at resolution time. Used to redirect logging implementations, replace deprecated artifacts, or use local project builds instead of published artifacts.

Q15. What does failOnVersionConflict() do?

Makes the build fail if any dependency has multiple conflicting versions in the resolution result. Useful for ensuring fully converged dependency trees.

Q16. What is dependency locking in Gradle?

Saves the exact resolved versions of all dependencies to lock files. Future builds use locked versions, preventing unexpected upgrades from dynamic versions or SNAPSHOT changes.

Q17. When should you use dependency locking?

For applications deployed to production — ensures exact same artifacts on every build. Less useful for libraries (would prevent consumers from getting dependency upgrades).

Q18. What is a dynamic version in Gradle?

A version range like 1.2.+ or [1.2,1.3) or latest.release/latest.integration. Gradle resolves these to a concrete version and caches the result (configurable duration with cacheDynamicVersionsFor).

Q19. What does cacheDynamicVersionsFor(0, 'seconds') do?

Forces Gradle to re-resolve dynamic versions on every build rather than using the cache. Useful for SNAPSHOT dependencies in active development.

Q20. How do you see the full dependency tree in Gradle?

./gradlew dependencies or for a specific config: ./gradlew dependencies --configuration runtimeClasspath. For why a specific dep was chosen: ./gradlew dependencyInsight --dependency groupId:artifactId.

Q21. What does the dependencyInsight task show?

Explains how and why a specific dependency version was selected: all paths to it, which version won the conflict, why it was selected over alternatives.

Q22. How do you exclude a dependency globally in Gradle?

configurations.all { exclude(group = 'commons-logging', module = 'commons-logging') }. This removes the dependency from all configurations.

Q23. What is a capability conflict in Gradle?

When two modules provide the same capability (e.g., both log4j and log4j-over-slf4j provide the 'log4j' capability). Gradle can detect and resolve these with capability conflict resolution rules.

Q24. How do you declare capabilities for conflict detection?

In a component metadata rule: allComponents { withCapabilities { addCapability('log4j', 'log4j', '0') } }. Then configure resolution: configurations.all { resolutionStrategy.capabilitiesResolution.withCapability('log4j:log4j') { ... } }.

Q25. What is a component metadata rule?

A rule that modifies or adds metadata for an existing published component — adding capabilities, fixing broken POMs, adding missing variants. Applied with dependencies { components { withModule(...) { ... } } }.

Q26. What is dependencyResolutionManagement in settings.gradle.kts?

Centralises repository declarations for all projects: dependencyResolutionManagement { repositories { mavenCentral() } }. Prevents each subproject from declaring repositories individually.

Q27. What is repositoriesMode in dependencyResolutionManagement?

PREFER_PROJECT (default) — project-level repositories override. PREFER_SETTINGS — settings repositories take precedence. FAIL_ON_PROJECT_REPOS — build fails if any project declares repositories directly.

Q28. How do you publish an artifact to Nexus/Artifactory with Gradle?

Apply the maven-publish plugin. Define a publishing { publications { create(...) { ... } } repositories { maven { url = ... credentials { ... } } } } block. Run ./gradlew publish.

Q29. What is the maven-publish plugin?

Generates and publishes POMs and artifacts to Maven repositories. Supports snapshot and release repositories, signing, and metadata generation.

Q30. What is the signing plugin?

Signs artifacts before publication using GPG. Required for publishing to Maven Central. Configure with signing { sign(publishing.publications) }.

Q31. What is Gradle's resolution metadata?

Gradle-specific metadata (Gradle Module Metadata, .module files) that provides richer information than Maven POMs — variant-aware publishing, capability declarations, rich version constraints.

Q32. What is variant-aware dependency resolution?

Gradle selects the appropriate variant of a dependency based on the consuming configuration's attributes (e.g., apiElements vs runtimeElements, jvmApiElements vs jvmRuntimeElements for Java).

Q33. What are attributes in Gradle's dependency resolution?

Key-value pairs that describe what a configuration needs or provides. Examples: org.gradle.usage (java-api, java-runtime), org.gradle.category (library, platform). Used for variant selection.